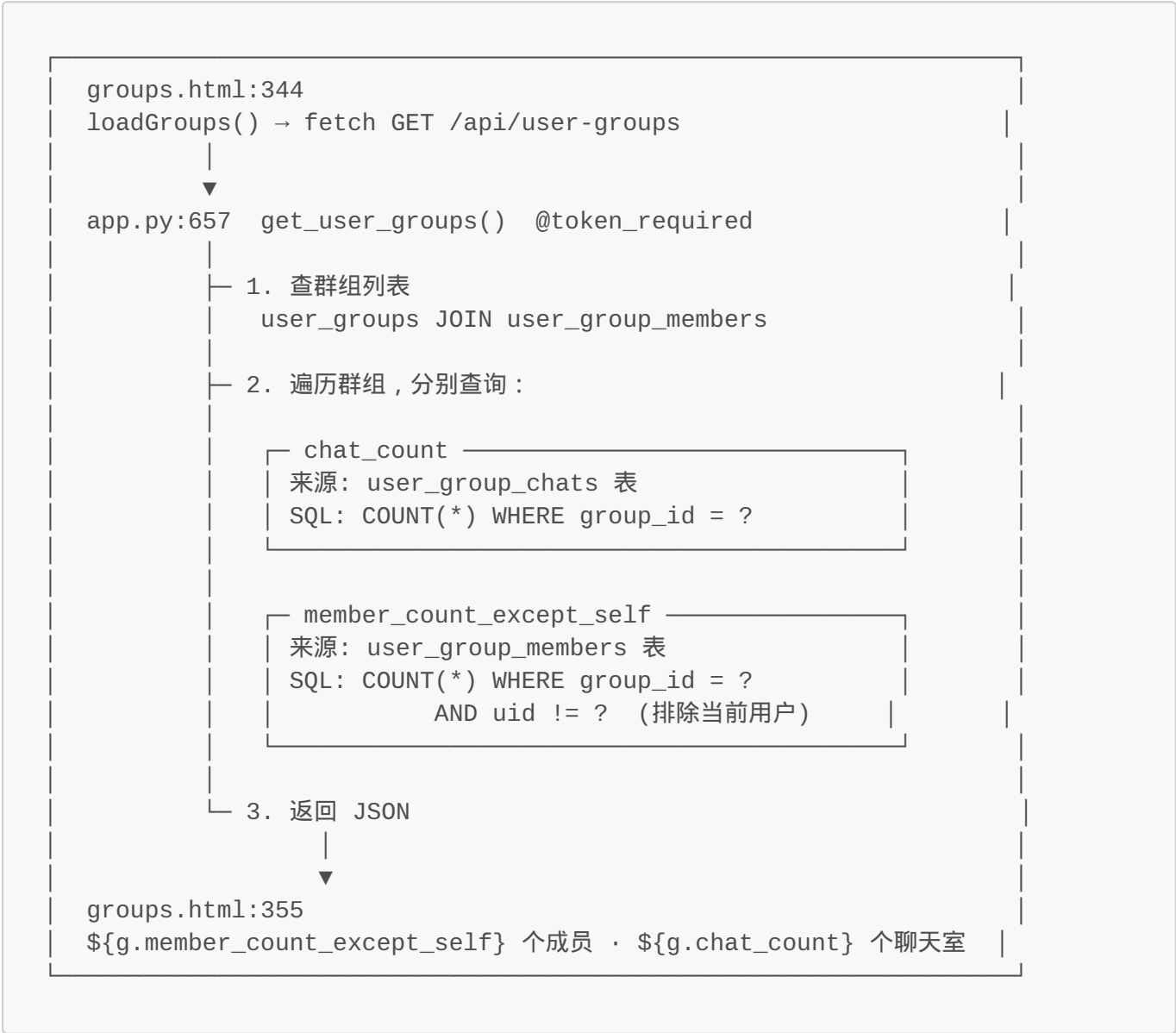


member_count_except_self 和 chat_count 数据流说明

概述

这两个字段出现在群组列表页，用于展示每个群组的成员数量和聊天室数量。它们由前端 `groups.html` 发起请求，后端 `app.py` 实时查询数据库计算得出。

数据流全景



```
async function loadGroups() {
  const groups = await api("/api/user-groups");
  const list = document.getElementById("groupList");
  list.innerHTML = groups.map(g => `
    <div class="group-item" data-gid="${g.group_id}" ...>
      ...
      <div class="item-sub">
        ${escapeHtml(g.category)} .
        ${g.member_count_except_self} 个成员 .
        ${g.chat_count} 个聊天室
      </div>
    </div>
  `).join("");
}
```

- 调用 `/api/user-groups` 接口 (GET 请求)
- 返回一个群组数组, 每个元素包含 `member_count_except_self` 和 `chat_count`
- 直接在模板字符串中渲染这两个值

2. 后端处理请求

文件: `app.py:657-687`

2a. 获取用户所有群组

```
groups = conn.execute('''
  SELECT g.* FROM user_groups g
  JOIN user_group_members gm ON g.group_id = gm.group_id
  WHERE gm.uid = ?
  ORDER BY g.created_at DESC
''', (uid,)).fetchall()
```

项目	说明
主表	<code>user_groups</code> (群组容器)
关联表	<code>user_group_members</code> (群组成员)
条件	<code>gm.uid = ?</code> — 只查当前用户所在的群组
排序	按创建时间倒序

2b. 遍历群组, 查询聊天室数量

```
chat_count = conn.execute('''
    SELECT COUNT(*) FROM user_group_chats WHERE group_id=?
''', (grp['group_id'],)).fetchone()[0]
```

项目	说明
表	<code>user_group_chats</code> — 群组与聊天室的关联表
字段	<code>group_id</code> 群组ID, <code>chat_id</code> 聊天室ID, <code>chat_name</code> 聊天室别名
SQL	统计该群组 ID 在关联表中的记录数
结果	该群组容器关联了多少个聊天室

2c. 遍历群组，查询成员数量（排除当前用户）

```
member_count = conn.execute('''
    SELECT COUNT(*) FROM user_group_members WHERE group_id=? AND uid!=?
''', (grp['group_id'], uid)).fetchone()[0]
```

项目	说明
表	<code>user_group_members</code> — 群组成员表
字段	<code>group_id</code> 群组ID, <code>uid</code> 用户ID, <code>added_at</code> 加入时间
SQL	统计该群组中 <code>uid ≠ 当前用户</code> 的记录数
结果	该群组中除了当前用户之外还有多少人

2d. 构造响应

```
result.append({
    'group_id': grp['group_id'],
    'name': grp['name'],
    'creator': grp['creator'],
    'created_at': grp['created_at'],
    'group_type': grp['group_type'],
    'category': grp['category'],
    'chat_count': chat_count,
    'member_count_except_self': member_count
})
return jsonify(result)
```

← 来自 2b
← 来自 2c

user_group_members — 群组成员

```
CREATE TABLE user_group_members (  
  group_id TEXT,      -- 群组ID, FK → user_groups.group_id  
  uid      TEXT,      -- 用户ID, FK → users.uid  
  added_at REAL,      -- 加入时间戳  
  PRIMARY KEY(group_id, uid)  
);
```

user_group_chats — 群组-聊天室关联

```
CREATE TABLE user_group_chats (  
  group_id TEXT,      -- 群组ID, FK → user_groups.group_id  
  chat_id  TEXT,      -- 聊天室ID, FK → groups_chat.group_id  
  chat_name TEXT,      -- 聊天室在此群组中的别名  
  PRIMARY KEY(group_id, chat_id)  
);
```

user_groups — 群组容器

```
CREATE TABLE user_groups (  
  group_id  TEXT PRIMARY KEY,  
  name      TEXT,  
  creator   TEXT,  
  created_at REAL,  
  group_type TEXT DEFAULT 'custom', -- 'custom' 或 'system'  
  category  TEXT DEFAULT '自定义'  -- '自定义'、'即时聊天'、'私聊'  
);
```

member_count_except_self 命名的设计意图

字段名中的 `_except_self` 表明：

- 这个计数排除了当前登录用户自身
- 前端显示如：3 个成员（当前用户看到"另外有3人"）
- 如果当前用户想看到包含自己的总数，需要 `member_count_except_self + 1`

注意事项

1. **无缓存**：每次调用 `/api/user-groups` 都实时执行两次 `COUNT(*)` 查询，群组数量多时可能有性能开销。
2. **N+1 查询问题**：当前实现对每个群组都单独执行两次查询（一次查 `chat_count`，一次查 `member_count_except_self`）如果有 N 个群组，总共需要 1（查群组列表）+ 2N 次查询。

3. **系统群组特殊处理**：在 `/api/user-groups/<id>/detail` 接口中（`app.py:775-846`），系统群组（即时聊天、私聊）的成员列表是动态计算的，但这**不影响**列表页的 `member_count_except_self`——列表页始终从 `user_group_members` 表直接统计。
4. **`chat_count` 只统计已关联的聊天室**：只有通过 `user_group_chats` 表与群组关联的聊天室才会计入，同一聊天室可能被关联到多个群组。